CLOUD-01: Cloud Integration Fundamentals

> **Series:** CLOUD | **Notebook:** 1 of 8 | **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

This notebook introduces how Dynatrace integrates with major cloud providers (AWS, Azure, GCP) to deliver full-stack observability across cloud environments. You will learn the difference between ActiveGate-based polling and API-based ingestion, how the Dynatrace entity model maps cloud resources, and how to plan a multi-cloud monitoring strategy.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`metrics.read`, `entities.read`, `ReadConfig`
Cloud Integration	At least one cloud provider connected (AWS, Azure, or GCP)
Prior Knowledge	Basic Dynatrace navigation and DQL fundamentals

1. Integration Architecture

Dynatrace supports multiple approaches for cloud integration, with **direct connections via the Clouds app** being the recommended modern approach:

Approach	Mechanism	Latency	Use Case
Clouds App (Direct Connection)	Dynatrace SaaS connects directly to cloud provider APIs — no ActiveGate needed	5 min	Recommended for SaaS: full entity discovery, topology, metrics
Classic ActiveGate Polling	Environment ActiveGate queries cloud provider APIs on a schedule	5-15 min	Legacy approach; required for Managed deployments
API-Based / Streaming	Cloud provider pushes data to Dynatrace via API (e.g., Firehose, Event Hub, Pub/Sub)	Near real-time	High-frequency metrics, log forwarding
OneAgent on Host	Agent installed directly on VMs/containers	Seconds	Deep code-level visibility, distributed tracing

How the Clouds App Works (Recommended)

1. Open the **Clouds app** in your Dynatrace environment.

2. Select the cloud provider (AWS, Azure, or GCP) and follow the guided onboarding flow.
3. For **AWS**: Dynatrace deploys a CloudFormation stack in your account that creates IAM roles, Secrets Manager entries, and optional log forwarding resources.
4. For **Azure**: Use the Azure Native Dynatrace Service from the Azure Marketplace, or connect via Entra ID app registration.
5. For **GCP**: Deploy the integration via Helm on a GKE cluster, using Pub/Sub for metric and log forwarding.
6. Dynatrace connects directly to cloud APIs — **no ActiveGate required** for SaaS deployments.
7. Discovered resources are mapped to the Dynatrace entity model, enriched with cloud-native metadata (tags, account IDs, regions).

> **Note:** Classic ActiveGate-based polling is still supported for Managed deployments and environments with strict network requirements, but direct connections are the recommended approach for SaaS.

Key Consideration: Integration vs Agent

Cloud integration (whether direct or via ActiveGate) provides **infrastructure-level** visibility (CPU, memory, network, cloud-specific metrics). For **application-level** visibility (distributed traces, code-level diagnostics), you still need OneAgent or OpenTelemetry instrumentation.

2. Connection Methods: Clouds App vs Classic

Clouds App — Direct Connections (Recommended)

The **Clouds app** provides a streamlined, fully managed experience for connecting cloud providers to Dynatrace SaaS. No customer-deployed ActiveGate is required.

Advantages:
- No ActiveGate infrastructure to deploy or maintain
- Guided onboarding with Infrastructure-as-Code (CloudFormation for AWS)
- Unified management interface for all cloud connections

- Automatic telemetry enrichment with cloud-native metadata (tags, account IDs)
- Supports metrics, logs, topology, and events from a single connection

Supported Providers:
Provider	Status	Onboarding Method
AWS	Generally Available	CloudFormation template via Clouds app
Azure	Available	Azure Native Dynatrace Service or Entra ID app
GCP	Available	Helm deployment on GKE with Pub/Sub

Classic ActiveGate-Based Polling (Legacy)

For Managed deployments or environments with specific network constraints, the classic ActiveGate approach is still available.

Advantages:
- Works with Dynatrace Managed (on-premises)
- Supports environments with outbound-only connectivity
- Single integration point behind firewalls

Limitations:
- Requires deploying and maintaining ActiveGate compute resources
- Polling interval introduces 5-15 minute data latency
- API rate limits can slow discovery in large environments
- Higher operational overhead

When to Use Which

Scenario	Recommended Approach
Dynatrace SaaS (new setup)	**Clouds app** direct connection
Dynatrace Managed	Classic ActiveGate polling
High-frequency alerting (< 5 min)	API-based streaming (Firehose, Event Hub, Pub/Sub)
Serverless workloads (Lambda, Functions)	Clouds app + Lambda Layer for tracing
Strict network isolation	Classic ActiveGate (outbound-only)

Scenario	Recommended Approach
Large-scale environments (1000+ resources)	Clouds app + selective streaming for critical metrics

3. Cloud Entity Model

Dynatrace maps cloud resources to its entity model. Each cloud resource becomes a monitored entity with relationships to other entities.

AWS Entity Types

Entity Type	Description
`dt.entity.ec2_instance`	EC2 virtual machines
`dt.entity.aws_lambda_function`	Lambda functions
`dt.entity.relational_database_service`	RDS instances
`dt.entity.elastic_load_balancer`	ALB/NLB/CLB load balancers
`dt.entity.aws_availability_zone`	Availability zones
`dt.entity.custom_device`	Other AWS services via CloudWatch

Azure Entity Types

Entity Type	Description
`dt.entity.azure_vm`	Virtual machines
`dt.entity.azure_web_app`	App Service instances
`dt.entity.azure_sql_database`	SQL Database instances
`dt.entity.azure_cosmos_db`	Cosmos DB accounts
`dt.entity.azure_load_balancer`	Load balancers

GCP Entity Types

Entity Type	Description
`dt.entity.cloud_application`	GKE workloads, Cloud Run services
`dt.entity.cloud_application_instance`	Individual pods/instances
`dt.entity.google_cloud_platform_service`	GCP managed services

4. Cloud Metrics vs Host Metrics

Understanding the difference between cloud-sourced metrics and OneAgent host metrics is critical for accurate monitoring.

Aspect	Cloud Metrics	Host Metrics (OneAgent)
Source	Cloud provider API (CloudWatch, Azure Monitor, etc.)	OneAgent on the host
Granularity	1-5 minutes	10-60 seconds
Scope	Infrastructure + managed services	OS, processes, code-level
Metric namespace	`cloud.aws.*`, `cloud.azure.*`, `cloud.gcp.*`	`dt.host.*`, `dt.process.*`
Cost	Cloud API call costs	No additional cloud cost
Coverage	All resources (even without agent)	Only instrumented hosts

Best Practice: Use Both

For compute resources (EC2, Azure VMs, GCE), use **both** cloud integration and OneAgent:
- Cloud integration provides the **cloud context** (instance type, availability zone, tags)
- OneAgent provides **deep observability** (processes, traces, code-level diagnostics)
- Dynatrace automatically correlates both data sources via the entity model

Let's query the cloud entities currently monitored in your environment.

5. Querying Cloud Entities

The following queries demonstrate how to explore cloud entities in your Dynatrace environment.

Count EC2 Instances

```
// Count all monitored EC2 instances
fetch dt.entity.ec2_instance
| summarize instance_count = count()

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes AWS_EC2_INSTANCE
// | summarize instance_count = count()
```

Count Azure VMs

```
// Count all monitored Azure VMs
fetch dt.entity.azure_vm
| summarize vm_count = count()
```

Compare Cloud Resources Across Providers

This query uses `append` to combine entity counts from multiple cloud providers into a single view.

```
// Compare cloud resource counts across providers
fetch dt.entity.ec2_instance
| summarize provider = "AWS", resource_type = "Compute (EC2)", resource_count
= count()
| append [
    fetch dt.entity.azure_vm
    | summarize provider = "Azure", resource_type = "Compute (VM)",
resource_count = count()
  ]
| append [
    fetch dt.entity.aws_lambda_function
    | summarize provider = "AWS", resource_type = "Serverless (Lambda)",
resource_count = count()
  ]
| sort provider asc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes AWS_LAMBDA_FUNCTION
// | summarize provider = "AWS", resource_type = "Compute (EC2)",
resource_count = count()
// | append [
// smartscapeNodes AWS_LAMBDA_FUNCTION
// | summarize provider = "Azure", resource_type = "Compute (VM)",
resource_count = count()
// ]
// | append [
// smartscapeNodes AWS_LAMBDA_FUNCTION
// | summarize provider = "AWS", resource_type = "Serverless (Lambda)",
resource_count = count()
// ]
// | sort provider asc
```

Host CPU Usage Across Cloud Hosts

This query retrieves average CPU usage for all monitored hosts, which includes both cloud and on-premises hosts.

```
// Average CPU usage across all hosts in the last hour
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

6. Multi-Cloud Strategy

When monitoring multiple cloud providers with Dynatrace, consider these best practices:

Naming Conventions

Establish consistent naming across providers:

Element	Convention	Example
Tags	`cloud-provider:aws`, `cloud-provider:azure`	Identify cloud origin
Management Zones	`Cloud - AWS - Production`, `Cloud - Azure - Staging`	Scope views by provider/env
Alerting Profiles	`cloud-aws-critical`, `cloud-azure-warning`	Route alerts by provider

Deployment Pattern

Component	Recommended Deployment
Environment ActiveGate	One per cloud provider, per region
OneAgent	On all compute instances (EC2, Azure VM, GCE)
Cloud Integration	Enabled per provider with least-privilege IAM
Log Forwarding	Centralized via OpenPipeline

Cost Considerations

- **Monitor only what you need** — disable unused service monitoring to reduce API calls
- **Use tag-based filtering** — monitor only tagged resources (e.g., `monitored:true`)
- **Review cloud API costs** — CloudWatch API calls, Azure Monitor queries, and GCP monitoring API calls all have costs
- **Right-size polling intervals** — not every metric needs 1-minute granularity

7. Summary and Next Steps

Key Takeaways

- Dynatrace integrates with cloud providers via the **Clouds app** (direct connections, recommended) or **classic ActiveGate polling** (legacy/Managed)
- Cloud resources are mapped to the **Dynatrace entity model** with automatic relationship discovery
- **Cloud metrics** complement **OneAgent host metrics** — use both for full visibility
- A consistent **naming convention** and **tagging strategy** is essential for multi-cloud governance

Next Steps

- **CLOUD-02: AWS Integration** — Deep dive into AWS-specific setup and monitoring
- **CLOUD-05: Azure Integration** — Azure Monitor integration and supported services
- **CLOUD-06: GCP Integration** — Google Cloud monitoring configuration

---